ZIWG CheatSheet – Laravel (1. ebaluazioa)

1. Komandoak

Komandoa	Zertarako
`composer create-project laravel/laravel izena`	Proiektu berria sortu
`composer require laravel/breeze --dev`	Breeze instalatu
`php artisan breeze:install blade`	Login, erregistroa eta logout sortu (Blade)
`npm install` · `npm run dev` · `npm run build`	Frontend-a (Vite, Tailwind)
`composer install`	`/vendor` berreskuratu.
`cp .env.example .env` · `php artisan key:generate`	`.env` eta aplikazioaren gakoa
`php artisan make:migration create_ekitaldiak_table`	Taula berria
`php artisan make:migration add_role_to_users_table --table=users`	Lehendik dagoen taulari zutabea gehitu
`php artisan migrate`	Migrazio berriak exekutatu
`php artisan migrate:rollback`	Azken migrazio-sorta desegin (`down()`)
`php artisan migrate:fresh --seed`	Taula guztiak ezabatu, berriro sortu eta seeder-ak exekutatu
`php artisan migrate:status`	Zein migrazio dauden exekutatuta
`php artisan make:seeder UserSeeder` · `php artisan db:seed`	Hasierako datuak
`php artisan make:factory EkitaldiaFactory`	Proba-datuen fabrika
`php artisan make:request StoreUserRequest`	Balidazioa klase propioan
`php artisan make:enum Enums/Role`	Enum-a (edo eskuz `app/Enums/Role.php`)
`php artisan storage:link`	`public/storage` → `storage/app/public` esteka

Komandoa	Zertarako
`php artisan route:list`	Ibilbide guztiak, izenak eta metodoak
`php artisan tinker`	PHP kontsola, modeloak probatzeko
`php artisan optimize:clear`	Cachea garbitu (config, route, view)

2. Fluxua (MVC) eta Breeze

Eskaera baten bidea:

1. **Route** (`routes/web.php`): URLa → kontrolatzailearen metodoa
2. **Middleware** (`auth` , rolak): baimena egiaztatu
3. **Controller** (`app/Http/Controllers`): balidatu, modeloari eskatu, bista itzuli
4. **Model** (`app/Models`): datu-basearekin hitz egin (Eloquent)
5. **View** (`resources/views/*.blade.php`): HTMLa, logikarik gabe

```
// routes/web.php
use App\Http\Controllers\UserController;

Route::middleware('auth')->group(function () {
    Route::resource('users', UserController::class);
});
```

Breeze-k sortzen duena: `routes/auth.php` (login, register, logout), `Auth/RegisteredUserController` , `ProfileController` , `resources/views/auth/*` , `x-app-layout` osagaia eta `dashboard` .

- Erabiltzaile logeatua: `auth()->user()` · `Auth::user()` · `auth()->id()` · `auth()->check()`
- Logout **POST** da beti, `@csrf` -arekin.
- Saioa hasi gabe sartzen bada, `auth` middleware-ak `/login` -era bidaltzen du.

3. Migrazioak

Migrazioak datu-basearen egitura definitzen du; `migrate` exekutatu arte ez da ezer aldatzen.

Zutabe-motak: `id()` · `string()` · `text()` · `integer()` · `unsignedInteger()` · `decimal(10,2)` · `boolean()` · `date()` · `dateTime()` · `foreignId()` · `timestamps()`

Modifikatzaileak: `->nullable()` · `->default(x)` · `->unique()` · `->after('zutabea')`

Gurasoa ezabatzean	Zer gertatzen da
`->cascadeOnDelete()`	Umeak ere ezabatzen dira
`->restrictOnDelete()`	Umeak badaude, ezin da ezabatu
`->nullOnDelete()`	Umeen FK-a NULL bihurtzen da (`->nullable()` behar du)

Ordena: gurasoaren taula (`kategoriak`) umearena (`ekitaldiak`) baino lehen sortu. Fitxategi-izenaren data-aurrizkiak ezartzen du ordena.

4. Eloquent modeloa eta erlazioak

Eloquent-ek ingelesezko pluralak asmatzen ditu: `Ekitaldia` → `ekitaldias` . Euskarazko izenekin, jarri `$table` beti.

`$fillable` : `create()` / `update()` -ek zerrenda honetako eremuak bakarrik gordetzen ditu. Eremu bat falta bada, ez da gordetzen eta ez da errorerik agertzen.

Oinarrizko metodoak

Kodea	Emaitza
`User::all()`	Guztiak
`User::latest()->paginate(10)`	Berrienak, orrikatuta (`{{ $users->links() }}`)
`User::find($id)`	Bat edo `null`
`User::findOrFail($id)`	Bat edo 404
`User::where('role', 'admin')->get()`	Iragazita
`User::where('email', $e)->first()`	Lehena edo `null`
`User::create($datuak)`	Sortu eta gorde
`$user->update($datuak)`	Eguneratu
`$user->delete()`	Ezabatu
`User::count()`	Kopurua

Erlazioak (FK-a beti "askoren" edo menpeko taulan)

Erlazioa	Alde batean	Bestean	FK / taula
1:1	`User` → `hasOne(Profila::class)`	`Profila` → `belongsTo(User::class)`	`profilak.user_id` + `unique()`
1:N	`Kategoria` → `hasMany(Ekitaldia::class)`	`Ekitaldia` → `belongsTo(Kategoria::class)`	`ekitaldiak.kategoria_id`
N:M	`User` → `belongsToMany(Ekitaldia::class)`	`Ekitaldia` → `belongsToMany(User::class)`	Pibota: `ekitaldia_user`

FK izena ez bada `modeloa_id`: `belongsTo(Kategoria::class, 'kat_id')`.

Metodoa vs propietatea: `$e->kategoria()` = query-a (gehiago kateatzeko) · `$e->kategoria` = emaitza.

5. Kontrolatzailea: CRUD eta balidazioa

Kontrolatzaileak koordinatzen du: balidatu → modeloari eskatu → bista edo redirect. Ez jarri HTML edo SQL gordinik hemen.

Route model binding: `User $user` parametroak `{user}` IDarekin bilatzen du erabiltzailea; ez badago, 404.

Balidazio-arau ohikoenak

Araua	Esanahia
`required` / `nullable`	Derrigorrezkoa / hutsik egon daiteke
`string` · `integer` · `numeric` · `boolean` · `date`	Mota
`min:8` · `max:255`	Luzera (testua) edo balioa (zenbakia) edo KB (fitxategia)
`email`	Email formatua
`unique:users,email`	Taulan errepikatu gabe
`Rule::unique('users')->ignore($user->id)`	Unique, editatzen ari den erregistroa izan ezik
`confirmed`	`password_confirmation` eremuarekin bat etorri
`exists:kategoriak,id`	FK-a existitzen da
`after:today`	Data etorkizunean

Araua	Esanahia
`in:admin,user` · `Rule::enum(Role::class)`	Balio onartuak
`image` · `mimes:jpeg,png,jpg,webp` · `max:2048`	Irudia, formatua, gehienez 2 MB

Balidazioak huts egiten badu, Laravelek automatikoki formulariora itzultzen du, `$errors` eta `old()` beteta.

Pasahitza editatzean: `nullable|min:8|confirmed`; hutsik badago, kendu `$datuak`-etik eguneratu aurretik.

6. Blade

Sintaxia	Zertarako
`{{ $aldagaia }}`	Inprimatu, ihes eginda (XSS-tik babestuta)
`{!! $html !!}`	Ihesik gabe: arriskutsua, saihestu
`@if` · `@elseif` · `@else` · `@endif`	Baldintzak
`@foreach ($users as $u)` … `@endforeach`	Begizta
`@forelse ($users as $u)` … `@empty` … `@endforelse`	Begizta + zerrenda hutsa
`@auth` … `@endauth` · `@guest` … `@endguest`	Logeatuta / ez
`@csrf`	CSRF tokena (POST/PUT/DELETE formulario guztietan)
`@method('PUT')` · `@method('DELETE')`	HTML-k GET/POST bakarrik ditu; hau simulatzeko
`@error('email')` … `{{ $message }}` … `@enderror`	Eremu baten errorea
`old('email', $user->email)`	Aurreko balioa edo lehenetsia
`route('users.edit', $user)`	Ibilbide-izenetik URLa
`asset('img/logo.png')`	`public/` karpetako fitxategia
`session('success')`	Flash mezua
`<x-app-layout>` … `</x-app-layout>`	Breeze-ren diseinua

Sintaxia	Zertarako
`$loop->iteration`	Begiztaren zenbakia (1etik)

7. Fitxategi igoera eta Storage

Irudia **diskoan** gordetzen da; datu-basean bere **bidea** soilik (testu laburra).

Formularioa → Kontrolatzailea (balidatu) → Diskoa (`storage/app/public`) → DB (bidea) → Bista (`Storage::url`)

Prestaketa (hiru pauso; bat ahaztuz gero irudiak ez dira agertzen)

1. Migrazioa: `$table->string('irudia')->nullable();` (hautazkoa, lehendik ekitaldiak daudelako) eta `php artisan migrate`
2. `.env` : `FILESYSTEM_DISK=public` eta `php artisan storage:link`
3. Modeloa: `'irudia'` `$fillable` -n

`accept="image/*"` erosotasuna da bakarrik: nabigatzailean saihestu daiteke.
Zerbitzariko balidazioa ezinbestekoa da.

Edit formularioan ere `enctype` + `@method('PUT')` + fitxategi-eremua jarri.

8. Rolak Enum bidez eta baimenak

Sei pauso, ordena honetan: Enum → Migrazioa → Modeloa → Erregistroa → Seeder → CRUD/bistak.

1. **Enum-a:** app/Enums karpetan Role izeneko enum bat sortu, string motakoa (backed enum). Bi kasu ditu: Admin (balioa "admin") eta User (balioa "user"). Rolen balio posibleak leku bakarrean geratzen dira.
2. **Migrazioa:** users taulari role izeneko testu-zutabe bat gehitu. Balio lehenetsia "user" da, beraz erabiltzaile berri guztiak arruntak dira.
3. **Modeloa:** User modeloan, role eremua bete-daitezkeen eremuen zerrendan (fillable) sartu. Gainera, casts atalean role Role enum-arekin lotu: Laravelek automatikoki bihurtzen du datu-baseko testua enum-era eta alderantziz.
4. **Seeder:** gutxienez erabiltzaile administratzaile bat sortu, rola Admin duela. Bestela, inork ez du kudeaketa egiteko baimenik izango.

Enum-aren erabilgarriak: `Role::Admin->value` (`'admin'`) · `Role::cases()` (guztiak, `<select>` bat betetzeko) · `Role::from('admin')` (stringetik Enum-era).

Botoi bat Blade-n ezkutatzeak ez du ezer babesten: URLa eskuz idatz daiteke.
Egiaztapena zerbitzarian ere egin behar da (kontrolatzailean edo middleware batean).

9. Akats ohikoak: sintoma → kausa

Sintoma	Kausa	Konponbidea
419 Page Expired	`@csrf` falta	Jarri `@csrf` formularioan
Formularioak funtzionatzen du, baina irudia ez da iristen	`enctype` falta	`enctype="multipart/form-data"`
Erregistroa sortzen da, eremu bat hutsik	Eremua ez dago `$fillable`-n	Gehitu `$fillable`-ra
DB-n bidea dago, irudia ez da ikusten (404)	`storage:link` falta	`php artisan storage:link`
Table 'ekitaldias' doesn't exist	Eloquent-ek ingelesez pluralizatu du	`protected $table = 'ekitaldiak';`
Migrazioak FK errorea	Umearen taula gurasoarena baino lehen	Migrazioen ordena aldatu
`show` / `edit`-en modeloa hutsik	Ibilbide-parametroa (`{ekitaldiak}`) eta aldagaia (`$ekitaldia`) ez datoz bat	`->parameters(['ekitaldiak' => 'ekitaldia'])` resource-ari
Editatzean "email already taken"	`unique` arauak bere burua aurkitzen du	`Rule::unique('users')->ignore($user->id)`
405 Method Not Allowed	PUT/DELETE simulatu gabe	`@method('PUT')` / `@method('DELETE')`
Call to a member function on null	Erlazioa `null` (adib. irudirik edo kategoriarik ez)	`@if` edo `?->` erabili
Diskoa fitxategi zaharrez betetzen da	Eguneratzean zaharra ez da ezabatu	`Storage::disk('public')->delete($bidea)`
Edonork egin dezake admin-aren lana	Rola bistan bakarrik egiaztatu da	Kontrolatzailean/middleware-an `abort(403)`
Kontrolatzaile luzeegia, HTML-arekin	MVC hautsita	Logika kontrolatzailean, HTMLa Blade-n

Entrega: ezabatu `/vendor` eta `/node_modules` zip-a egin aurretik. Berreskuratzeko: `composer install` + `npm install` + `npm run build`.